

TrailSense Multilateration Library: CLI Reference

July 12, 2026

CONTENTS

TrailSense Multilateration Library: CLI Reference

CLI reference: trailsense-triangulate

Synopsis

Input

anchors[]

obs[]

mac_hash format

Parser limits

Output

Exit status

Stderr

Relationship to the C structs

TrailSense Multilateration Library: CLI Reference

The trailsense-triangulate command-line interface contract: input JSON, output JSON, exit status, and diagnostics. Part of the TrailSense Multilateration Library documentation set.

CLI reference: trailsense-triangulate

The stdin/stdout/exit contract of the `trailsense-triangulate` command.

Synopsis

```
trailsense-triangulate < input.json > output.json
```

One JSON request from stdin, one JSON response to stdout. No arguments, options, or environment variables. Reads stdin to EOF before processing.

Input

A single JSON object with two required array keys, `anchors` and `obs`. Unknown top-level keys skipped. Trailing data after the top-level object is an error.

```
{
  "anchors": [ ... ],
  "obs":     [ ... ]
}
```

ANCHORS[]

Each element a surveyed sensor-node position; object non-empty.

Field	Type	Required	Default	Units / notes
<code>node_id</code>	integer	yes	none	must fit int32
<code>lat</code>	number	yes	none	degrees
<code>lon</code>	number	yes	none	degrees
<code>enrolled</code>	boolean	no	<code>true</code>	<code>false</code> excludes this anchor
<code>path_loss_a_cdb</code>	integer	no	<code>0</code>	centi-dB; <code><= 0</code> means “use band default”
<code>path_loss_n_milli</code>	integer	no	<code>0</code>	milli; <code><= 0</code> means “use band default”

OBS[]

Each element one node’s sighting of one device; object non-empty. All five fields required.

Field	Type	Required	Units / notes
<code>node_id</code>	integer	yes	must fit int32
<code>mac_hash</code>	string	yes	exactly 8 hex chars (4 bytes); case-insensitive on input
<code>band</code>	integer	yes	wire range <code>0..3</code> accepted; <code>0</code> =wifi, <code>1</code> =ble, <code>2</code> =cell; band <code>3</code> is parsed but rejected at solve time
<code>rsi</code>	integer	yes	dBm; must fit int8 (<code>-128..127</code>)
<code>ts_ms</code>	integer	yes	must fit uint32 (<code>0..4294967295</code>)

MAC_HASH FORMAT

8 hex chars (lower or upper case) mapping to 4 bytes: `"deadbeef"` becomes `de ad be ef`. Any length other than 8, or any non-hex character, is an error.

PARSER LIMITS

At most 256 anchors and 8192 observations per request; exceeding either is a fatal input error (nonzero exit). JSON nesting limited to 64 levels. Distinct from the core solver caps under STDERR below.

Output

On success, a single JSON object with one key, `positions`, an array. Empty when no group resolved: `{"positions": []}`. Each element:

Field	Type	Units / notes
<code>mac_hash</code>	string	8 lowercase hex chars
<code>band</code>	integer	<code>0</code> =wifi, <code>1</code> =ble, <code>2</code> =cell
<code>signal_type</code>	string	"wifi", "bluetooth", or "cellular" (band 0/1/2)
<code>fingerprint_hash</code>	string	band prefix (<code>w_</code> / <code>b_</code> / <code>c_</code>) + the 8 hex <code>mac_hash</code> , e.g. "w_deadbeef"
<code>lat</code>	number	degrees, 9 decimal places
<code>lon</code>	number	degrees, 9 decimal places
<code>accuracy_m</code>	number	weighted CEP, meters; 2-node fix equals <code>error_semi_major_m</code>
<code>measurement_count</code>	integer	participating anchors (not post-outlier survivors)
<code>confidence</code>	integer	<code>40</code> (exactly 2 anchors), <code>70</code> (exactly 3), <code>85</code> (4 or more)
<code>peak_rssi</code>	integer	strongest participating rssi, dBm
<code>error_semi_major_m</code>	number	semi-major axis, meters
<code>error_semi_minor_m</code>	number	semi-minor axis, meters
<code>error_major_axis_deg</code>	number	bearing of semi-major axis (0=North, clockwise, folded into <code>[0,180)</code>)

`signal_type` and `fingerprint_hash` are CLI-added, not in `ts_position_t`. All other fields correspond one-to-one with the struct.

Exit status

Code	Meaning
0	Success. JSON written to stdout. Includes the zero-positions case (<code>{"positions": []}</code>) and the case where a cap warning was emitted on stderr.
non-zero	Input error. A diagnostic on stderr; nothing usable on stdout.

Input errors: stdin not readable, malformed JSON, missing required field, empty anchor or obs object, `mac_hash` not exactly 8 hex chars, non-finite number, `node_id` / `rss_i` / `ts_ms` out of range, `band` outside `0..3`, more than 256 anchors or 8192 obs, or trailing data after the top-level object. Never emits a partial or guessed position on bad input.

Stderr

Diagnostics only, never JSON. Two purposes:

- **Fatal errors** (nonzero exit): one line prefixed `trailsense-triangulate: input error:` or `trailsense-triangulate: ts_triangulate failed`.
- **Cap warnings** (exit 0): the core silently drops input past its static caps; the CLI detects this, warns, but still emits the partial result and exits 0. Two caps trigger a warning:
 - more than `TS_MAX_GROUPS` (64) distinct `(mac_hash, band)` groups, and
 - more than `TS_MAX_NODES_PER_GROUP` (32) distinct nodes within one group.

Nonempty stderr with exit 0 means “partial result, some groups/nodes dropped,” not failure. Distinguish by exit code, not by whether stderr is empty.

Relationship to the C structs

Field semantics match the C structs exactly: `anchors[]` maps to `ts_anchor_t`, `obs[]` to `ts_obs_t`, each `positions[]` element to `ts_position_t` (plus CLI-only `signal_type` and `fingerprint_hash`). For field-level semantics including the mixed 2-node error-ellipse meaning, see the C API Reference and Output Reference.